

Exercicio 1: Construción da imaxe de produción para Laravel

Explicación Agora que temos xa o arquetipo de Laravel, vamos coller xa un código implantado para ver como podemos realizar a posta en produción dunha aplicación realizada con este *framework*. Deberemos tamén lanzar unha migración de base de datos, debido a que a aplicación necesita un esquema concreto para o seu funcionamento.

1. Crea un `fork` do proxecto `Arquetipo de Laravel`

e clónao no teu equipo de nome `T05.02 Despregue Laravel`

.

Descarga o seguinte [código](#) e inclúeo no directorio `src` (copia ben tódolos ficheiros oculos, senón podes obter problemas).

Comproba que podes levantar o proxecto sen problemas con `make up`

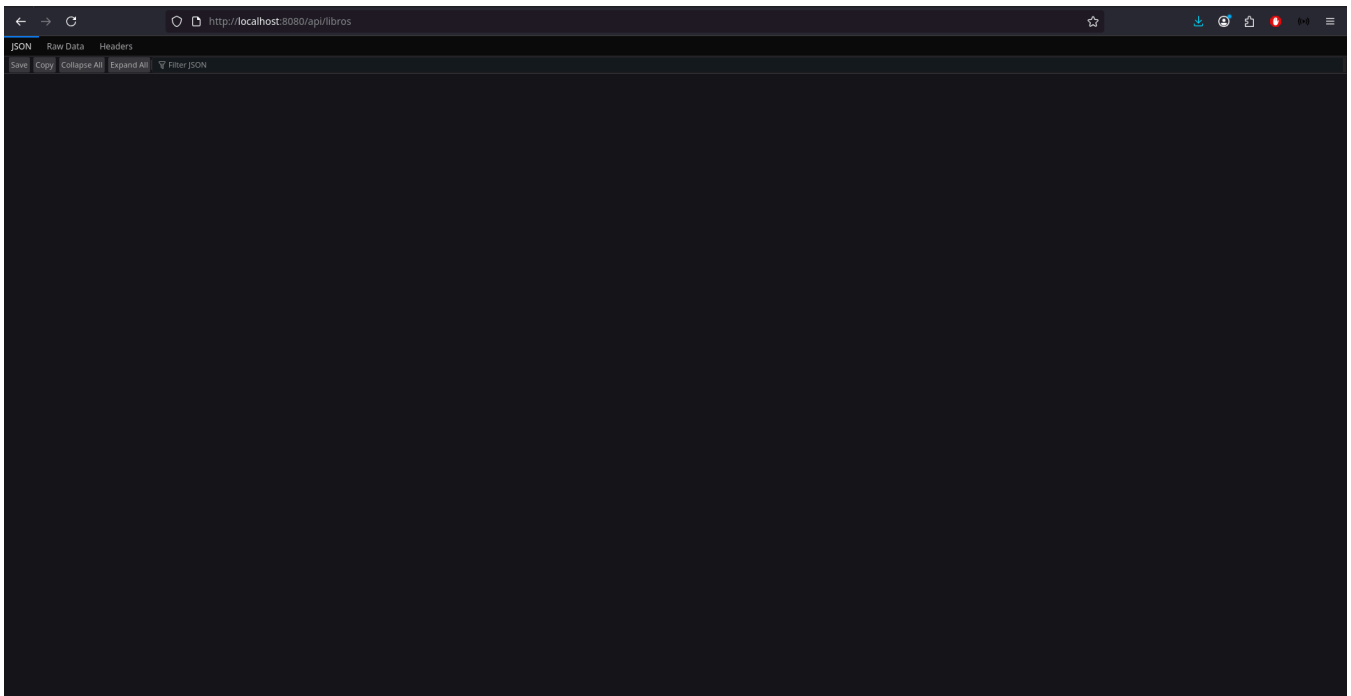
visitando `localhost:8080`

.

Realiza a migración. Despois visita a URL para comprobar que non hai erros

`http://localhost:8080/api/libros`

e que a aplicación funciona correctamente. **Realiza capturas** do funcionamento da aplicación.



Elimina o directorio `vendor`

1. do directorio `src`.

Explicación Agora procederemos a poñer desenvolver o procedemento de posta en produción da aplicación. Esta posta en produción realizarémola en Docker.

1. Crea o `Dockerfile`

de produción de nome `Dockerfile.prod`

. Recorda instalar as librerías necesarias para conectar con PostresSQL.

Xera unha `APP_KEY`

para o proxecto (esta inclúe a parte de `base64`

1.) utilizando o terminal:

```
APP_KEY="base64:$(head -c 32 /dev/urandom | base64)"
echo $APP_KEY
```

1. Crea o o ficheiro `docker-compose.prod.yaml`

e `entrypoint.prod.sh`

1. para produción. Modifica tan só as variables de conexión a base de datos polas que tiñamos en Aiven.
Mapea o servizo da aplicación para que escoite polo **porto 9001**.

Explicación Podemos crear novos comandos `make` para poder probar a posta en produción da nosa aplicación. Neste caso as probas podémolas facer no propio equipo e comprobar que todo funciona correctamente.

1. Crea un novo comando `deploy`

para despregar en produción e outro `deploy-stop`

1. para parar a produción:

```
# E interesante o --build para que se reconstrúa a imaxe sempre
deploy:
  docker compose -f docker-compose.prod.yaml up -d --build

deploy-stop:
  docker compose -f docker-compose.prod.yaml down
```

1. Pon en marcha a aplicación en produción e comproba que funciona a URL: <http://localhost:9001/api/libros>.
Entrega capturas dos ficheiros `Dockerfile.prod`

, `docker-compose.prod.yaml`, `entrypoint.prod.yaml` e `Makefile`

```

🐳 DockerFile.prod
1  #### Etapa 1: Builder ####
2  FROM composer:2.6 AS builder
3
4  WORKDIR /app
5
6  # Copiamos ficheros de dependencias e instalamos
7  COPY src/ ./
8  RUN composer install --no-dev --optimize-autoloader --no-scripts
9
10
11  #### Etapa 2: Imaxe final con Apache ####
12  FROM php:8.2-apache
13
14  # Instalar extensiones necesarias para Laravel
15  RUN apt-get update && apt-get install -y \
16      libzip-dev \
17      zlib-dev \
18      libpq-dev \
19      && docker-php-ext-install zip pdo_pgsql pgsql \
20      && rm -rf /var/lib/apt/lists/*
21
22  # Directorio de traballo
23  WORKDIR /var/www/html
24
25  # Copiar aplicación desde o builder
26  COPY --from=builder /app /var/www/html
27
28  # Cambiar permisos para usuario non root
29  RUN chown -R www-data:www-data /var/www/html
30
31  # Copiamos a definición do virtualhost
32  COPY laravel.conf /etc/apache2/sites-available/000-default.conf
33
34  # Habilitar mod_rewrite de Apache
35  RUN a2enmod rewrite
36
37  # Copiamos o entrypoint
38  COPY entrypoint.prod.sh /usr/local/bin/entrypoint.sh
39  RUN chmod +x /usr/local/bin/entrypoint.sh
40  ENTRYPOINT ["/usr/local/bin/entrypoint.sh"]
41
42  RUN echo "ServerName localhost" >> /etc/apache2/apache2.conf
43
44  # Cambiamos ao usuario de Apache
45  USER www-data
46
47  # Comando por defecto: Apache en primeiro plano
48  CMD ["apache2-foreground"]

```

🐳 docker-compose.prod.yaml

```

1  services:
2    app:
3      build:
4        context: .
5        dockerfile: DockerFile.prod
6      environment:
7        APP_ENV: production
8        APP_DEBUG: false
9        APP_KEY: kgVDA0ptBa76DCftNh4vyRQ+517kiwQpxQEmg6BpsE=
10       SESSION_DRIVER: file
11       DB_CONNECTION: pgsql
12       DB_HOST: pg-36a04b08-iessanclemente-864d.j.aivencloud.com
13       DB_PORT: 10378
14       DB_DATABASE: defaultdb
15       DB_USERNAME: avnadmin
16       DB_PASSWORD: AVNS_RdVZOaSiUWzeIuEGGjv
17     restart: always
18     ports:
19       - "9001:80"
20     networks:
21       - laravel_prod
22
23  networks:
24    laravel_prod:

```

```

$ entrypoint.prod.sh
1  #!/bin/bash
2  set -e
3
4  APP_DIR="/var/www/html" # directorio raíz de Apache no contenedor
5  cd "$APP_DIR"
6
7  # Copiar .env.example a .env se non existe
8  if [ ! -f ".env" ]; then
9      cp .env.example .env
10 fi
11
12 # Sobrescribir tódalas variables de contorno relevantes no .env
13 # Filtra só variables típicas de Laravel
14 env | while IFS='=' read -r key val; do
15     case "$key" in
16         APP_*|DB_*|SESSION_*|CACHE_*|QUEUE_*|MAIL_*|LOG_*)
17             if grep -q "^$key=" .env; then
18                 sed -i "s|^$key=.*|$key=$val|" .env
19             else
20                 echo "$key=$val" >> .env
21             fi
22         ;;
23     esac
24 done
25
26 # Creamos directorios que Laravel necesita e non sempre crea
27 mkdir -p storage/framework/cache \
28         storage/framework/sessions \
29         storage/framework/views \
30         bootstrap/cache
31
32 # Limpar cache de configuración para asegurar que Laravel use as variables actualizadas
33 php artisan config:clear
34 php artisan cache:clear
35 php artisan view:clear
36
37 # Creamos as novas caches de configuración
38 php artisan config:cache
39 php artisan route:cache
40 php artisan view:cache
41
42 # Finalmente, deixar que Apache arranque
43 exec "$@"

```

```

M Makefile
1  # Variables. Define o nome do contedor da aplicación. Por iso é importante o valor container_n
2  APP = laravel_app
3
4  ##### Tarefas
5
6  # Levantar contedores. Executa "docker compose up" en modo segundo plano (`-d`). Levanta tódol
7  up:
8      docker compose up -d
9
10 # Parar contedores. Para e elimina os contedores levantados.
11 down:
12     docker compose down
13
14 # Entra no contedor da aplicación. Abre unha sesión interactiva ("bash") dentro do contedor `l
15 bash:
16     docker exec -it $(APP) bash
17
18
19 # Executar migracións. Executa as migracións de Laravel dentro do contedor. Mantén a base de d
20 migrate:
21     docker exec -it $(APP) php artisan migrate
22
23
24 # Instalar unha librería específica. para executar: "make install-lib LIB_NAME=monolog/monolog
25 install-lib:
26     docker exec -it $(APP) composer require $(LIB_NAME)
27
28 # E interesante o --build para que se reconstrúa a imaxe sempre
29 deploy:
30     docker compose -f docker-compose.prod.yaml up -d --build
31
32 deploy-stop:
33     docker compose -f docker-compose.prod.yaml down

```

1..

2. Para o despregue, e agora sube ao repositorio tódolos cambios.

Explicación Podemos engadir os ficheiros xerados para a posta en produción ao noso arquetipo de Laravel. Deste xeito, cando creemos unha nova aplicación con este *framework* tamén teremos realizado o procedemento de posta en produción.

1. Agora no repositorio `Arquetipo de Laravel`

podes engadir o novo `Makefile` e os ficheiros de `Dockerfile.prod`, `docker-compose.prod.yaml` e `entrypoint.prod.yaml`

No repositorio `Arquetipo de Laravel`

crea un ficheiro `Readme.md` explicando en que consiste o arquetipo na súa totalidade: que contén, como funciona, para que serve, etc. **Entrega capturas** da totalidade do ficheiro `Readme.md`

```

1  README.md > ## # Arquetipo de Laravel > ## Para que sirva
2  # Arquetipo de Laravel
3
4  Este repositorio contém un **arquetipo de aplicación Laravel**.
5
6  ## Contido
7
8  O arquetipo inclúe:
9
10 - Estrutura base dun proxecto Laravel.
11 - Configuración para conexión a PostgreSQL.
12 - Ficheiros para poata en produción (Todos os .prod).
13 - Makefile con comandos útiles.
14
15 ## Como funciona
16
17 1. **Desenvolvemento**
18    - Levanta os contedores en modo desenvolvemento.
19    - Podes acceder á aplicación en 'http://localhost:8080'.
20
21 2. **Producción**
22    - Modifica 'docker-compose.prod.yaml' coas credenciais de base de datos reais.
23    - Levanta a aplicación de produción.
24    - Podes acceder á aplicación en 'http://localhost:9001'.
25
26 ## Para que sirva
27
28 Pódese empregar como base para outros proxectos de laravel, so tendo que ambiar a información dentro dos docker compose sobre a base de datos empregada.

```

1..

Exercicio 2: Despregue da aplicación de Laravel en produción

Explicación Vamos preparar a nosa máquina de produción (`dapw_iniciais_server_01`) para poder utilizar os ficheiros `Makefile`

. Tamén instalaremos Docker, debido a que agora nesta máquina a produción realizarase mediante esta tecnoloxía.

1. Na máquina `dapw_iniciais_server_01`

recupera a instantánea `admin_remota_configurada`

.

Instala o paquete `build-essential`

para poder utilizar os ficheiros `Makefile`

.

Instala Docker: <https://docs.docker.com/engine/install/debian/>. E realiza a configuración post instalación (<https://docs.docker.com/engine/install/linux-postinstall>). E dicir, engade o usuario `dadmin`

ao grupo `docker`. **Entrega captura** do `Hello world` de Docker. (Non é necesario `Docker rootless`

)

```

dadmin@iniciaisserver01:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:05813aedc15fb7b4d732e1be879d3252c1c9c25d885824f6295cab4538cb85cd
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Apaga a máquina e crea unha nova instantánea `docker_instalado`

. Crea ademais unha `OVA` de nome `dapw-docker.ova`

1. . É importante esta última porque necesitarémola máis adiante.

Explicación A partir das seguintes tarefas utilizaremos varias máquina e vai ser importante ter claro que IP ten cada unha delas. Polo tanto é conveniente non utilizar o servidor DHCP para isto, e utilizar IPs fixas nos servidores. Senón dependeríamos ata da secuencia de inicio das máquinas para que se repetisen as IPs.

Esta máquina a partir deste momento converterase no **servidor de produción con Docker**.

1. Deshabilita o servidor DHCP da rede `Anfitrión sólo anfitrión`

Inicia a máquina `dapw_iniciais_server_01`

e modifica o ficheiro `/etc/network/interfaces` para asignar unha IP estática a máquina na rede de `Anfitrión sólo anfitrión`

1.:

```

auto enp0s8
iface enp0s8 inet static
address 192.168.56.101
netmask 255.255.255.0

```

1. Executa o comando `sudo systemctl restart networking.service`

1. e comproba con `ip a` que obtiveches a IP desexada.

Explicación Para finalizar a tarefa, tan só necesitamos o noso repositorio para despregar en produción a nosa aplicación. Docker permítenos reproducir de xeito exacto a posta en produción. Polo tanto se funcionou no noso equipo, vai funcionar en calquera servidor.

1. Inicia a máquina `dapw_iniciais_server_01`

. Clona o repositorio `T05.02 Despregue Laravel`

e executa o despregue.

Proba que todo funciona utilizando a seguinte URL: `http://192.168.56.101:9001/api/libros`

. **Entrega captura** onde se vexa a aplicación funcionando.

